# DesignGrad: Conceptual, Numerical and Visual Feedback for a Local CAD Coding Agent

**Sinan Candan** · Kaggle Gemma 4 Developer Agent Paper Track

**Abstract.** Mechanical design is an iterative process. This paper studies which kinds of textual feedback improve the designs of a local language model that writes parts as CAD code. **DesignGrad** is inspired by TextGrad: a critic reviews the current design, and the review is converted into instructions for revising it. Gemma 4 26B-A4B runs offline on a laptop and performs every role in the loop. Three kinds of feedback are added cumulatively: conceptual feedback, in which the critic reads the design document; numerical feedback, in which the agent writes and runs its own analysis code to measure the built part and to calculate quantities such as mass, stress and deflection; and visual feedback, in which a reviewer inspects rendered views of the built part. After the loop has finished, independent tests score each design against its requirements. On three design problems with three repeats each, the score increases with every level of feedback, by 28% in total, and the largest increase comes from numerical feedback. The three kinds of feedback also detect different faults: conceptual feedback identifies details that the design document leaves undefined, numerical feedback identifies the requirements that the built part does not meet, and visual feedback often identifies the cause, for example a plate built in the wrong orientation.

## 1 Introduction

Mechanical design is an iterative process. Engineers begin with a brief problem statement, develop it into a detailed design, model the part and evaluate the result. Each evaluation leads to a revision, and the cycle continues until the part meets its requirements.

As AI moves towards the physical world, agents that can do mechanical design are becoming popular [1–5]. Feedback-driven improvement is well established in software engineering: coding agents perform better when they execute their code, read the errors and test the output [6–9]. Design agents have adopted the same principle, using rendered images of the part [3, 4], geometric measurements from the CAD kernel [5] and finite-element analysis in the loop [10, 11]. However, the contribution of each kind of feedback has not been measured separately. Consequently, there is little evidence to guide the design of a harness for design agents.

DesignGrad is inspired by TextGrad [12], a method for the automatic improvement of prompts and outputs. In TextGrad, a critic evaluates an output, and the evaluation is propagated backwards as a "textual gradient": an instruction for revising the text that produced the output. DesignGrad applies this method to two texts, a design document and a CAD program.

The feedback levels are inspired by how engineers work: they iterate on their design. They receive a problem statement with little detail and develop it into a design document. They review this document to verify that the concept satisfies every requirement. They model the part and analyse the model with measurements and simulations. Finally, they inspect a rendering of the finished part to confirm that it matches their intent and that every feature is in place. We give a local agent these three evaluations as conceptual, numerical and visual feedback, and ask: **how do its designs improve as conceptual, numerical and visual feedback are added, and how should a local model be used for design?**

The system runs offline on a laptop with one open model, Gemma 4 26B-A4B [13]. For simple parts, local operation makes design agents accessible and inexpensive, keeps design data on the user's machine, and allows deployment on the local computers that are connected to manufacturing equipment. The designs are scored after the loop has finished, by independent tests that are not part of the feedback.

## 2 The system

### 2.1 The loop

DesignGrad is a TextGrad-inspired loop [12] that iteratively improves two texts: a **design document**, which states the concept, the dimensions and how each requirement is satisfied, and the **CadQuery program** that builds the part. One Gemma 4 model performs every role in the loop (Fig. 1). The designer reads the specification and writes both texts. The CAD agent executes the program and builds the part. The critic evaluates the design against each requirement and returns a verdict per requirement together with a list of specific problems, each stating the feature, its current value and the required value (Fig. 11). A gradient step converts the critique into revision instructions for the design document and for the program, and the designer rewrites both. The loop runs for at most four rounds.

At the end of each round, the new design is compared with the best design so far, using the critic's verdicts and the agent's own measurements. The next round continues from the new design if it is better, and otherwise restarts from the best design.

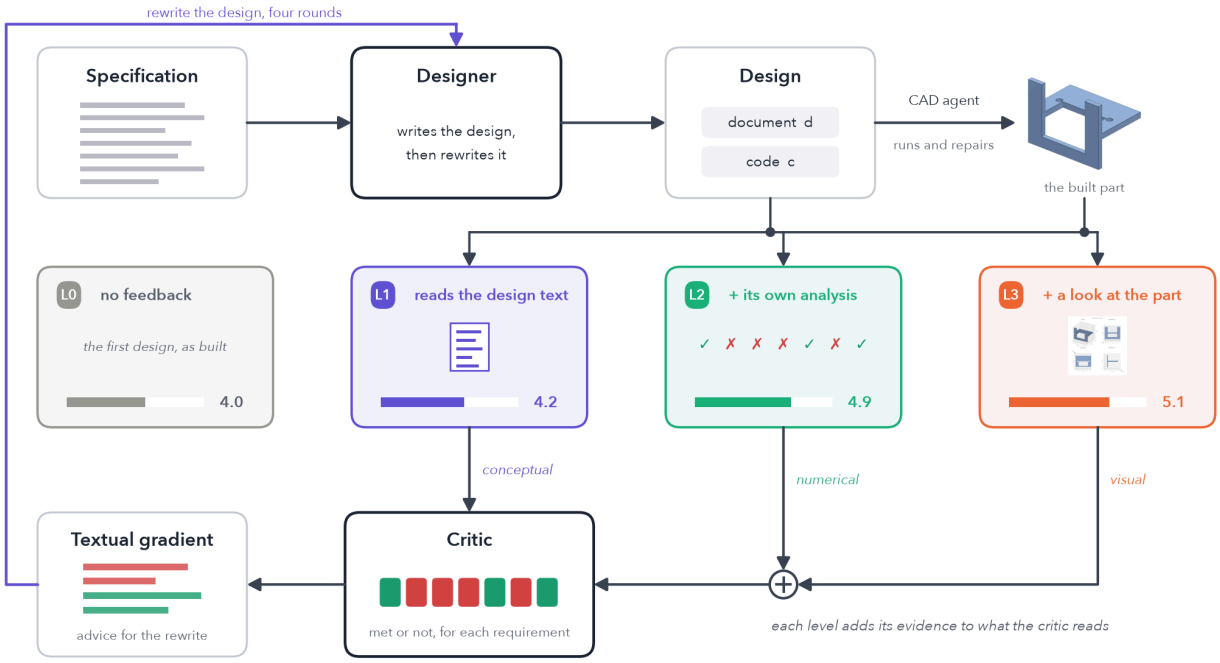



**Fig. 1.** The DesignGrad loop: design, build, review, textual gradient, revision. Bars: requirements met (of 7) at each level.

### 2.2 Feedback levels

The four levels differ only in the information that the critic receives. They are cumulative: each level retains the feedback of the levels below, and all start from the same first design.

**L0, no feedback.** The designer writes the design document and the CadQuery program once, and the CAD agent builds the part. No critic is involved and the design is not revised.

**L1, conceptual feedback.** The critic receives only the design document and does not see the built part. For each requirement, the critic recalculates masses, stresses and positions from the dimensions stated in the document, and reports every statement that is incorrect and every dimension that is not defined.

**L2, numerical feedback.** The critic additionally receives measurements of the built part. An analysis agent writes a Python program for each problem from the specification. The prompt is identical for all problems and contains only general reference material (library calls and textbook formulas). The program loads the built part and evaluates every requirement. It probes the geometry (whether material is present at a given point, whether a given region is empty), measures dimensions and mass, and, for the shelf bracket, applies beam theory to the cross-section at the wall to obtain bending stress and deflection. Because a self-written check can be incorrect,

each analysis program is also executed on a control part, a one-millimetre cube without features (Fig. 2). A check that reports a hole or a wall on the cube does not measure the intended feature and must be rewritten; a measurement that fails this control is not accepted as evidence.

**L3, visual feedback.** The critic additionally receives a visual review. A reviewer inspects four rendered views of the built part together with the design document and lists every difference between the part and the document.

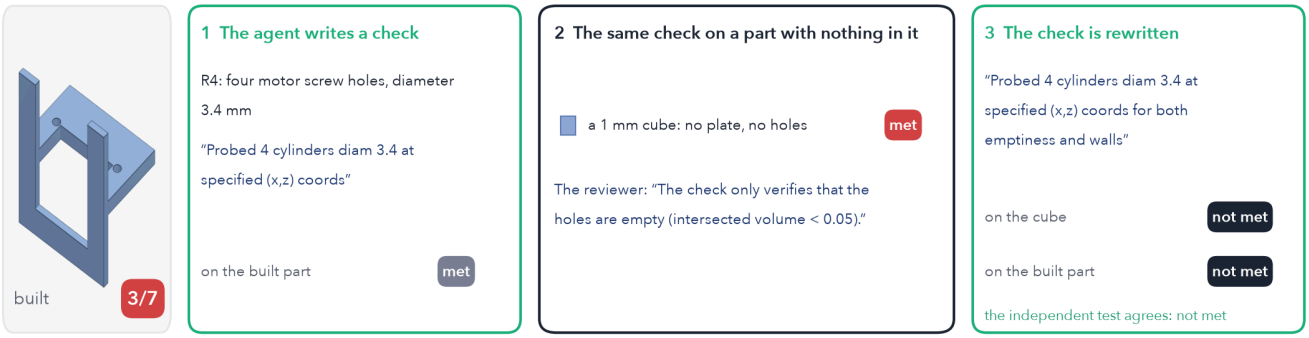



**Fig. 2.** The control part at L2 (motor mount). The first check for the motor screw holes reports them on a part whose face plate is missing, and also on the empty cube; the rewritten check is correct.

## 3 Experiment

**Problems.** The main experiment uses three parts, each with seven numerical requirements (Fig. 4): a shelf bracket that must carry a load, an electronics enclosure base, and a motor mount. Each part is run three times with different sampling seeds, giving nine runs per level.

**Independent tests.** For each problem, a set of geometric and analytic tests, one per requirement, was written separately from the agents' tools and fixed before any run. A reference design passes all tests and a plain block fails most of them. They are applied after a run has finished. "Requirements met" always denotes the number of these tests that a design passes.

**Model.** `gemma4:26b` in Ollama on an M1 Max laptop with 32 GB of memory, temperature 0.3.

## 4 Results

### 4.1 Results by level

**Table 1. Final designs, nine runs per level.** Best design produced: the best of any round, retained or not.

| Level | Requirements met (of 7) | Best design produced | Model calls | Minutes |
|---|---|---|---|---|
| L0 no feedback | 4.0 | 4.0 | 2 | 1 |
| L1 conceptual | 4.2 | 5.0 | 14 | 4 |
| L2 + numerical | 4.9 | 5.1 | 39 | 14 |
| L3 + visual | **5.1** | **5.7** | 44 | 15 |

Each level of feedback improves the design (Table 1, Fig. 3). Conceptual feedback alone has a small effect. Numerical feedback produces the largest single increase, 0.7 requirements, and visual feedback adds a further 0.2. From L0 to L3 the designs gain 1.1 requirements, an improvement of 28% (95% interval +0.4 to +1.9); five of nine runs improve and none deteriorates. Each single step lies within the variation between runs, whereas the overall gain does not.

Within the loop (Fig. 3, top right), the retained design improves steadily at L2 and L3, whereas at L1 it is unstable, because a critic that cannot observe the built part sometimes prefers a revision whose part is worse.

The three parts respond differently (Fig. 3, bottom; Fig. 4). The enclosure base does not change at any level: one first design already meets all seven requirements, and the other two fail only the requirement on the screw bosses,

which no revision corrects, so the points in Fig. 3 coincide. The shelf bracket improves when numerical feedback is added (3.3 requirements at L0 and L1, 4.7 at L2, 4.3 at L3); the mass limit, the envelope and screwdriver access most often remain unmet. The motor mount improves at every level, most with visual feedback (2.3, 3.0, 3.7, 4.7).

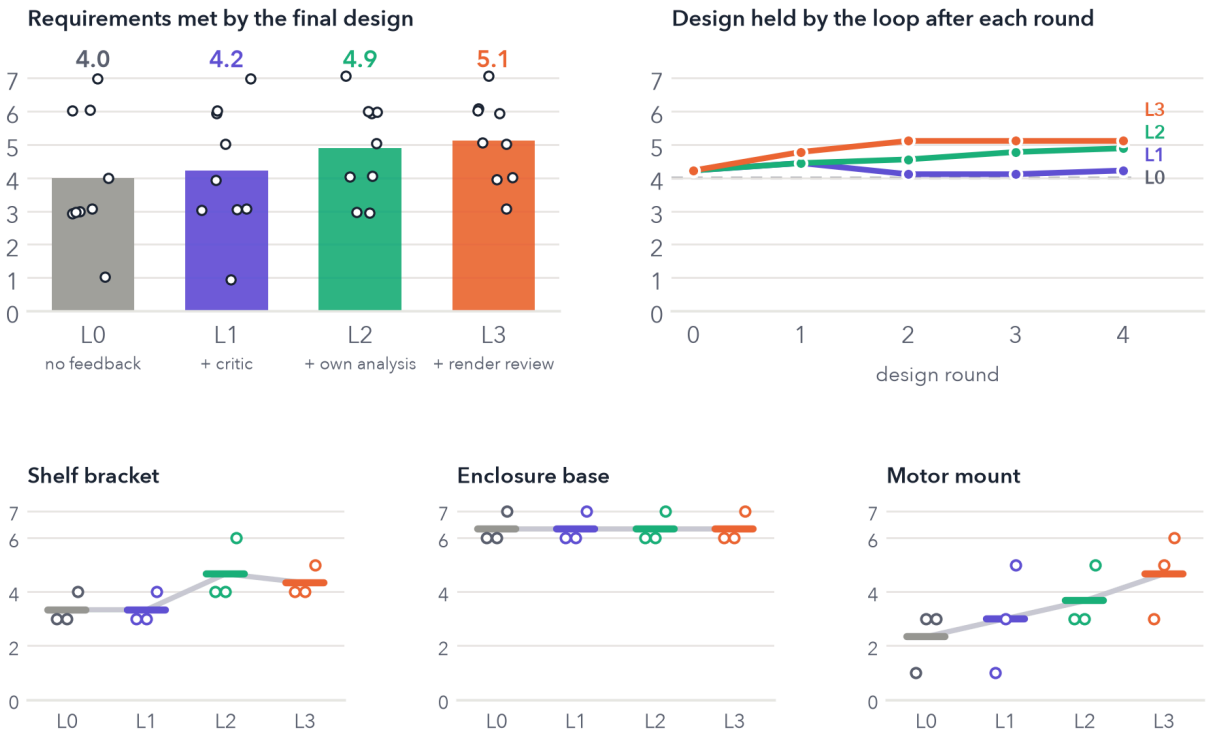



**Fig. 3.** Top left: requirements met by the final design; each point is one run. Top right: the design held by the loop after each round, mean of nine runs (dashed: L0; the curves start higher because one first design was repaired further). Bottom: each part; points are repeats, bars are means.

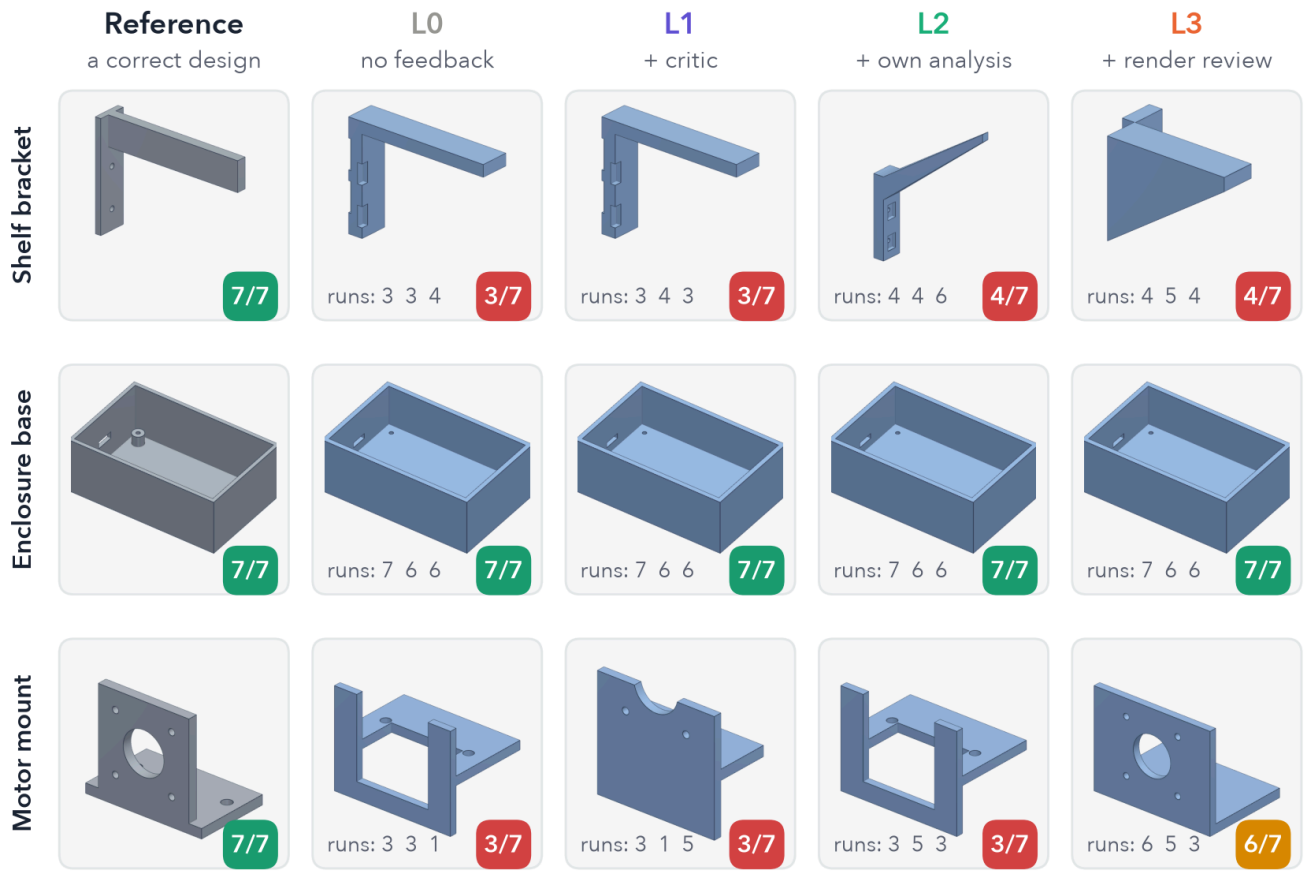



**Fig. 4.** A correct reference design and the final design of each level, first repeat; "runs" lists all three repeats. **Shelf bracket:** two wall-screw holes with screwdriver access, a seat 110 mm up and 150 mm out, 150 N within stress and deflection limits, at most 90 g. **Enclosure base:** an 85 × 55 mm tray, four screw bosses, a USB-C cutout, at most 70 g. **Motor mount:** a 4 mm face plate with a bore and four screw holes, motor clearance, a base with two bolt holes, at most 60 g.

### 4.2 Three critics, one design

Fig. 5 shows the three critics evaluating the same first design: a motor mount whose face plate was built in the wrong plane and whose base lies below the table.




**Fig. 5.** The same first design at each level (first repeat): the part before, what the critic read and wrote, the change in the design document, and the part after. Badges: requirements met. Quotes in all figures are verbatim.

At L1 the critic cannot detect this fault, because the design document describes the face plate correctly. The critic instead identifies the undefined dimensions of the clearance pocket, and these are added. The document improves, but the built part does not.

At L2 the analysis program finds neither the bore nor the screw holes at their specified positions, and the critic reports this. The report is correct but describes the symptoms and not the cause. The designer replaces the plate with a solid block, the part remains incorrect, and four rounds pass without progress.

At L3 the reviewer inspects the rendered views and identifies both faults directly. After one revision the part meets six requirements. Numerical feedback indicated what was wrong; visual feedback indicated why.

### 4.3 Round by round

When a measurement identifies the fault, numerical feedback is effective (Fig. 6, top; Fig. 7). The first shelf bracket weighs 445 g against a limit of 90 g; the designer replaces the solid section at the wall with a hollow section, and the next part weighs 43 g and meets six requirements. The motor mount (Fig. 6, bottom; Fig. 8) improves similarly, but the loop does not retain the best part, produced in its last round (Section 4.4).




**Fig. 6.** Two loops, round by round: the part of each round, the requirements it meets, whether the loop retained it, and the first problem the critic named. Framed: the final design.

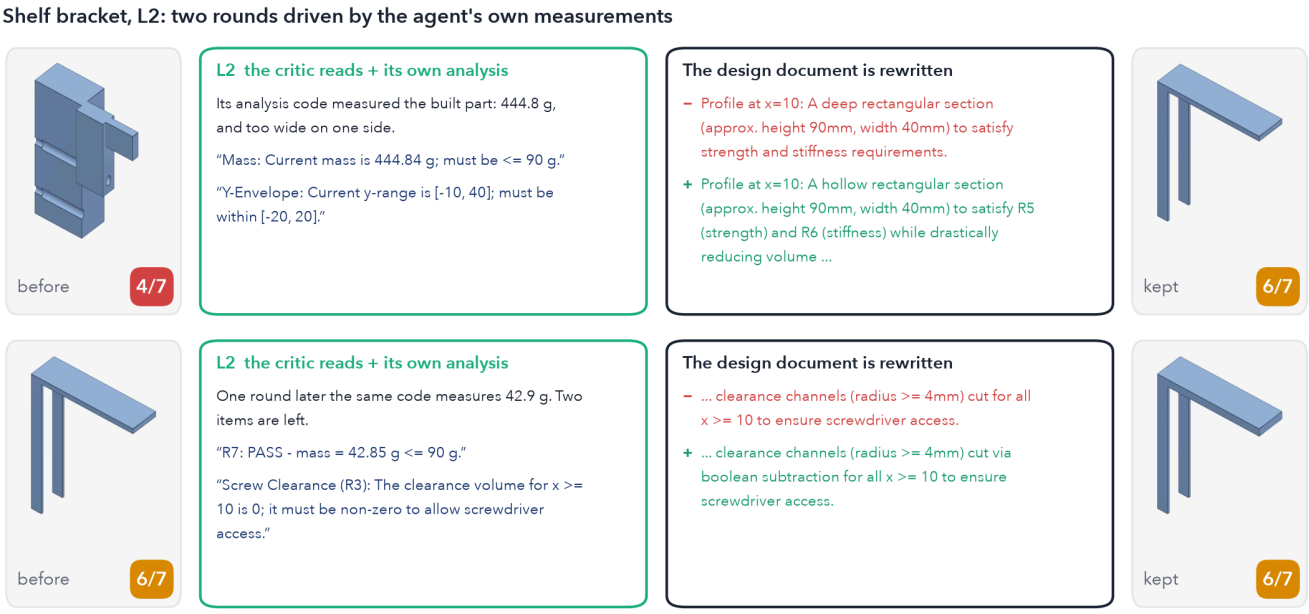



**Fig. 7.** The first two rounds of the shelf bracket.

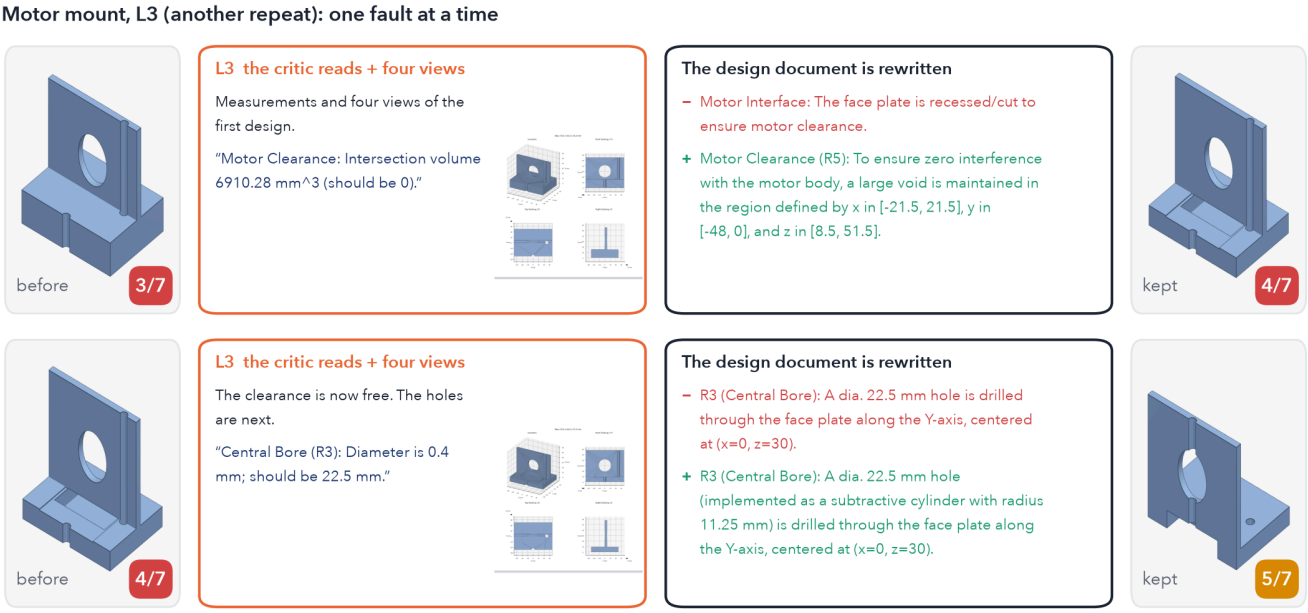



**Fig. 8.** The first two rounds of the motor mount.

### 4.4 What limits the loop

Three factors limit the loop (Fig. 9).

The first is the revision step. One design round corrects approximately one unmet requirement in seven at L1 and L2 and one in three at L3, and breaks approximately one in ten of the requirements that were already met.

The second is the agent's own checks. After the control run they rarely miss a real fault: of 207 unmet requirements, they reported 201. However, approximately one verdict in ten reports a met requirement as unmet. At L2 and L3 the loop produced nine designs that were better than the design it held and did not retain them; eight of the nine carried such a false alarm. The best motor mount in Fig. 6 was lost in this way.

The third is the visual reviewer. A quarter of the reviewer's remarks that name a requirement concern one that the part already meets. Visual feedback is the most informative and the least reliable, and is therefore most useful together with measurements.

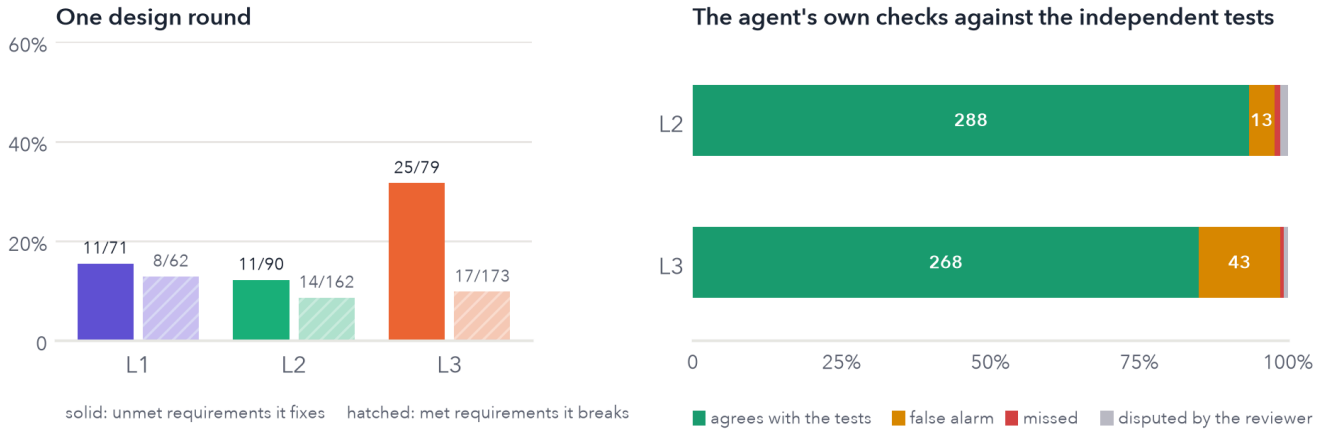



**Fig. 9.** Left: the effect of one design round. Right: the agent's own verdicts compared with the independent tests.

### 4.5 Other parts

Four further parts were run twice each (Fig. 10): a phone stand, a pipe clamp, a drone arm and a heat sink (six requirements). None of them was used when the loop was revised (Section 7). The trend is similar: the designs meet 3.9 requirements at L0, 3.6 at L1, 4.6 at L2 and 4.7 at L3; three runs improve from L0 to L3 and none deteriorates. Two runs are excluded because the CAD agent did not complete most of their builds, so their scores

do not measure the feedback: a tilted sensor mount (12 of 16 builds; 0, 2, 2, 2 from L0 to L3) and the second pipe clamp run (10 of 16; 2, 4, 3, 5).




**Fig. 10.** All seven parts: a correct reference, requirements met at each level (mean of the repeats), and the part retained at L3.

## 5 Discussion

The results suggest three recommendations for using a local model for design. First, the agent should build and test the part: a critic that reads only the design document improves the document, while the built part barely changes. Second, the agent should measure the part and verify its own measurements, since a control part exposes checks that measure nothing. Third, the agent should inspect the part: rendered views often reveal the cause of a fault where measurements reveal only the symptom. The remaining difficulty is to remove a fault without breaking another requirement and to decide reliably whether a new part is better. Further gains are expected from the revision step, the comparison of candidate designs, and testing the part in a physics simulation.

## 6 Related work

Self-feedback, code execution and self-written tests improve model outputs [6–9], although self-critique without external evidence is unreliable [14]. CAD agents refine their code from rendered views [3, 4], kernel measurements [5] and finite-element analysis [10, 11]. DesignGrad differs in three respects: the model writes its own analysis code, this code is verified on a control part, and feedback is added cumulatively so that each contribution can be measured.

## 7 Limitations

Claude wrote the problems and the independent tests and assisted with the code; fixing the tests before the runs and validating them on reference and incorrect designs reduces the risk of bias but does not remove it. The sample is small: nine runs per level, with large variation between runs, and two runs per further part. The loop was revised after first passes over the three main parts, so these parts were not unseen; all reported runs use the final version. The analysis is geometric and analytic, not finite-element.

## 8 Reproducibility

```
python -m designgrad.run --ladder v6 --problems P01 --levels L0 L1 L2 L3 --out runs/x
python eval/v4_analyze.py runs/x --out runs/x
```

Every model call is recorded; replaying the records reproduces the runs we checked byte for byte. Code, problems, tests and records will be released.

**Critic**

```
Review the design. Write your answer in three parts with exactly
these headings.

## Working
Short notes, requirement by requirement: do your arithmetic and
weigh the evidence here. You may change your mind here.

## Verdicts
Exactly one line per requirement R1..Rn, in the form "Rk:
PASS|FAIL|UNSURE - reason" with the numbers you rely on. These are
final: one verdict per line, no second thoughts, nothing else in
this part.

## Problems
The problems to fix, most severe first. Be to the point: for each
problem name the feature, the value that is wrong and the value it
should have. No general advice. If nothing needs fixing, write
"None".
```

**Reviewer of the agent's checks: the control run**

```
The same script was also run on a control part: a 1 mm cube in
place of the real part, so almost nothing is there. On the control
part it reported:
- pass: R1, R4, R7
- fail or error: R2, R3, R5, R6
A 1 mm cube has no plate, wall, hole, bore, boss, slot or surface.
If a requirement asks for a feature to EXIST, or for material to
be somewhere, and its check still passes on the cube, then the
check only tests that a region is empty; it is NOT faithful. It
must also measure the material the feature belongs to (for a hole:
the opening is empty AND the material around it is solid).
Requirements that only set a limit (mass, envelope, a keep-out or
clearance zone, a single valid solid) are rightly met by the cube;
do not mark those unfaithful for this reason. Take the control run
into account in your JSON answer. This control rule concerns ONLY
the requirements listed under "pass" above. A check that fails or
errors on the cube has passed the control: judge it on the five
points alone. A check that reports FAIL on the real part is not
unfaithful merely because its value is surprising or contradicts
the design document: the part may really be wrong there.
```

**Critic at L1: nothing but the design document**

```
None beyond the design document. Do not take the document's own
claims on trust. In your working, recompute every number it claims
from the dimensions it states (mass from volume and density,
stresses, deflections, positions, clearances, sizes) and check
that the stated features and coordinates really satisfy each
requirement. If the document does not state the dimensions needed
to decide a requirement, the verdict is UNSURE and the missing
dimension is a problem to fix.
```

**Analysis agent: two of its rules**

```
- A hole, bore, slot or cutout is a feature IN material. Check
  both: the opening is empty AND the material it passes through is
  there (filled fraction of a ring or shell just outside the
  opening). An empty region alone does not show a hole: empty
  space has no holes.
- Compare measured values with limits using a small tolerance
  (0.01 mm for lengths, 0.1 % otherwise): CAD kernels return
  values such as 20.0000001 for 20.
```

**Fig. 11.** Four of the prompts, verbatim.

## References


[1] Y. Liu et al. Aligning cyber space with physical world: a comprehensive survey on embodied AI. arXiv:2407.06886, 2024.

[2] L. Zhang et al. Large language models for computer-aided design: a survey. arXiv:2505.08137, 2025.

[3] K. Alrashedy et al. Generating CAD code with vision-language models for 3D designs. ICLR, 2025.

[4] X. Li et al. Seek-CAD: a self-refined generative modeling for 3D parametric CAD using local inference via DeepSeek. ICLR, 2026.

[5] J. Barkley, R. Loghmani and A. Barati Farimani. CADSmith: multi-agent CAD generation with programmatic geometric validation. arXiv:2603.26512, 2026.

[6] A. Madaan et al. Self-Refine: iterative refinement with self-feedback. NeurIPS, 2023.

[7] N. Shinn et al. Reflexion: language agents with verbal reinforcement learning. NeurIPS, 2023.

[8] X. Chen et al. Teaching large language models to self-debug. ICLR, 2024.

[9] B. Chen et al. CodeT: code generation with generated tests. ICLR, 2023.

[10] E. Berger et al. Physics-in-the-Loop: a hybrid agentic architecture for validated CAD engineering design. IJCAI-ECAI, 2026.

[11] G. Son et al. Self-improving CAD generation agents with finite element analysis as feedback. arXiv:2605.17448, 2026.

[12] M. Yuksekgonul et al. Optimizing generative AI by backpropagating language model feedback. *Nature* 639, 609–616, 2025.

[13] Gemma Team. Gemma 4 technical report. arXiv:2607.02770, 2026.

[14] J. Huang et al. Large language models cannot self-correct reasoning yet. ICLR, 2024.